

UACM

Universidad Autónoma
de la Ciudad de México

Nada humano me es ajeno

Academia de Informática Cuauhtpec Matutino

Ingeniería de Software

Construcción y evolución del software

Laboratorio de Ingeniería de Software

Profesor: Emmanuel Vite Sevilla

Nombre de la práctica: Programa uno

Número: 3

Alumno(s)/Equipo:

Axel Javier Rios Sánchez

Evelin Maldonado Espinosa

Jesús Josué Suarez López

Mauricio Charbel Chavez Galindo

Fecha: 07/04/2026



Introducción

La evolución del proyecto de este simulador del juego UNO refleja una transición profunda desde una programación estructurada básica hacia una arquitectura profesional basada en Programación Orientada a Objetos (POO). En las primeras versiones, el código presentaba una lógica lineal y rígida, donde una sola clase intentaba gestionar todas las reglas, validaciones y comportamientos de las cartas, lo que resultaba en un sistema difícil de mantener y propenso a errores. A través de las sucesivas mejoras, se implementaron conceptos avanzados como la abstracción, permitiendo que las cartas y los jugadores posean identidades propias y comportamientos especializados.

Esta transformación no solo mejoró la estética visual en la consola, sino que optimizó la escalabilidad y robustez del software. La introducción de clases abstractas para los jugadores permitió integrar una inteligencia artificial (CPU) de forma transparente, mientras que el uso de estructuras de datos como Stacks y LinkedLists aseguró un manejo de memoria eficiente y un flujo de juego ininterrumpido. Al delegar las responsabilidades a métodos específicos y encapsular la lógica interna, el sistema pasó de ser un script funcional a un motor de juego dinámico, capaz de manejar excepciones, reglas complejas y múltiples participantes con total fluidez.

Evidencia del código 2

Universidad Autónoma
de la Ciudad de México
NADA HUMANO ME ES AJENO

```

import java . util . * ;

clase Carta {
    color de la cadena ;
    Cadena valor ;

    static final String RESET = " \u 001B[0m " ;
    static final String ROJO = " \u 001B[31m " ;
    static final String VERDE = " \u 001B[32m " ;
    static final String AMARILLO = " \u 001B[33m " ;
    static final String AZUL = " \u 001B[34m " ;
    static final String BLANCO = " \u 001B[37m " ;

    public Carta ( String color , String valor ) {
        este . color = color ;
        este.valor = valor ;
    }

    Cadena colorANSI ( ) {
        interruptor ( color ) {
            caso " Rojo " : devolver ROJO ;
            caso " Verde " : devolver VERDE ;
            caso " Azul " : devolver AZUL ;
            caso " Amarillo " : return AMARILLO ;
            predeterminado : devolver BLANCO ;
        }
    }

    cadena pública [ ] dibujar ( ) {
        Cadena c = colorANSI ( ) ;
        devolver nueva cadena [ ] {
            c + " +-----+ " + REINICIAR ,
            c + " | " + Cadena . formato ( " %-8s " , valor ) + " | " + REINICIAR ,
            c + " | | " + REINICIAR ,
            c + " | " + Cadena . formato ( " %-8s " , color ) + " | " + REINICIAR ,
            c + " | | " + REINICIAR ,
            c + " | " + Cadena . formato ( " %8s " , valor ) + " | " + REINICIAR ,
            c + " +-----+ " + REINICIAR
        } ;
    }
}

```

Evidencia del código 3

```

import java.util.*;

// --- ABSTRACCIÓN ---
abstract class Carta {
    protected String color;
    protected String valor;

    public static final String RESET = "\u001B[0m";
    public static final String ROJO = "\u001B[31m";
    public static final String VERDE = "\u001B[32m";
    public static final String AZUL = "\u001B[34m";
    public static final String AMARILLO = "\u001B[33m";
    public static final String BLANCO = "\u001B[37m";

    public Carta(String color, String valor) {
        this.color = color;
        this.valor = valor;
    }

    public String getColor() { return color; }
    public String getValor() { return valor; }

    public abstract boolean esCompatible(Carta otra, String colorActual);
    public abstract void aplicarEfecto(Juego contexto);

    protected String getANSI() {
        return switch (color) {
            case "Rojo" -> ROJO;
            case "Verde" -> VERDE;
            case "Azul" -> AZUL;
            case "Amarillo" -> AMARILLO;
            default -> BLANCO;
        };
    }

    public String getLinea(int linea) {
        String c = getANSI();
        String vDibujo = valor.equals(anObject: "Color") ? "comodin" : valor;
        String[] diseño = {
            c + "+-----+" + RESET,
            c + "| " + String.format(format: "%-8s", vDibujo) + " |" + RESET,
            c + "| " + RESET,
            c + "| " + String.format(format: "%-8s", color) + " |" + RESET,
            c + "| " + RESET,
            c + "| " + String.format(format: "%8s", vDibujo) + " |" + RESET,
            c + "+-----+" + RESET
        };
    }
}

```

```

        return diseño[linea];
    }
}

// --- CLASES CONCRETAS ---
class CartaNumerica extends Carta {
    public CartaNumerica(String color, String valor) { super(color, valor); }
    @Override public boolean esCompatible(Carta otra, String colorActual) {
        return this.color.equals(colorActual) || this.valor.equals(otra.getValor());
    }
    @Override public void aplicarEfecto(Juego contexto) {}
}

class CartaAccion extends Carta {
    public CartaAccion(String color, String valor) { super(color, valor); }
    @Override public boolean esCompatible(Carta otra, String colorActual) {
        return this.color.equals(colorActual) || this.valor.equals(otra.getValor());
    }
    @Override public void aplicarEfecto(Juego contexto) {
        switch (valor) {
            case "Salto" -> contexto.saltarTurno();
            case "Rev" -> contexto.reversaEfecto();
            case "+2" -> contexto.hacerRobar(n: 2);
        }
    }
}

class CartaComodin extends Carta {
    public CartaComodin(String valor) { super(color: "Negro", valor); }
    @Override public boolean esCompatible(Carta otra, String colorActual) { return true; }
    @Override public void aplicarEfecto(Juego contexto) {
        if (valor.equals(anObject: "+4")) contexto.hacerRobar(n: 4);
        contexto.cambiarColorGlobal();
    }
}
}

```

NADA HUMANO ME ES AJENO

El código se mejoró mediante la implementación de pilares fundamentales de la Programación Orientada a Objetos, transformando una estructura plana en una arquitectura robusta y escalable. Al declarar Carta como una clase abstracta, se impide la creación de cartas genéricas sin sentido, obligando a definir comportamientos específicos. Se utilizó el pilar de la herencia para especializar el comportamiento de cartas numéricas, de acción y comodines, lo que elimina la necesidad de condicionales complejos y repetitivos en la lógica principal. El uso del otro pilar como el polimorfismo se destaca en los métodos esCompatible y aplicarEfecto, permitiendo que el juego trate a cualquier carta de forma genérica mientras cada una ejecuta su propia lógica interna. También se mejoró lo que es el encapsulamiento mediante modificadores de acceso protected, protegiendo los datos internos. Esta nueva estructura facilita enormemente la adición de nuevas

reglas o tipos de cartas sin modificar el código existente, aplicando así el principio de responsabilidad única y mejorando la legibilidad y el mantenimiento a largo plazo.

Evidencia del código 2

```
class Baraja {
    List < Carta > cartas = new ArrayList <> ( ) ;

    público Baraja ( ) {
        crear ( ) ;
        Colecciones . barajar ( cartas ) ;
    }

    void crear ( ) {
        String [ ] colores = { " Rojo " , " Verde " , " Azul " , " Amarillo " } ;
        para ( String color : colores ) {
            cartas . add ( nueva Carta ( color , " 0 " ) ) ;
            para ( int i = 1 ; i <= 9 ; i ++ ) {
                cartas . add ( new Carta ( color , " " + i ) ) ;
                cartas . add ( new Carta ( color , " " + i ) ) ;
            }
            // Implementación de cartas especiales: Reversa, Salto, Roba 2
            para ( int i = 0 ; i < 2 ; i ++ ) {
                cartas . add ( nueva Carta ( color , " +2 " ) ) ;
                cartas . add ( nueva Carta ( color , " salto " ) ) ;
                cartas . add ( nueva Carta ( color , " rev " ) ) ;
            }
        }
        // Implementación de cartas especiales: Roba 4 y Comodín
        para ( int i = 0 ; i < 4 ; i ++ ) {
            cartas . add ( nueva Carta ( " Negro " , " +4 " ) ) ;
            cartas . add ( nueva Carta ( " Negro " , " comodín " ) ) ;
        }
    }

    Carta repartir ( ) {
        if ( cartas . isEmpty ( ) ) return null ;
        devolver cartas.remove ( cartas.size ( ) - 1 ) ;
    }
}
```

Evidencia del código 3

NADA HUMANO ME ES AJENO

```
// --- CLASE BARAJA ---
class Baraja {
    private LinkedList<Carta> mazo = new LinkedList<>();
    private Stack<Carta> descarte = new Stack<>();

    public Baraja() {
        String[] colores = {"Rojo", "Verde", "Azul", "Amarillo"};
        for (String c : colores) {
            mazo.add(new CartaNumerica(c, valor: "0"));
            for (int i = 1; i <= 9; i++) {
                mazo.add(new CartaNumerica(c, String.valueOf(i)));
                mazo.add(new CartaNumerica(c, String.valueOf(i)));
            }
            for (int i = 0; i < 2; i++) {
                mazo.add(new CartaAccion(c, valor: "Salto"));
                mazo.add(new CartaAccion(c, valor: "Rev"));
                mazo.add(new CartaAccion(c, valor: "+2"));
            }
        }
        for (int i = 0; i < 4; i++) {
            mazo.add(new CartaComodin(valor: "Color"));
            mazo.add(new CartaComodin(valor: "+4"));
        }
        Collections.shuffle(mazo);
    }

    public Carta robar() {
        if (mazo.isEmpty()) {
            if (descarte.size() <= 1) return null;
            Carta cima = descarte.pop();
            mazo.addAll(descarte);
            descarte.clear();
            descarte.push(cima);
            Collections.shuffle(mazo);
        }
        return mazo.pollFirst();
    }

    public void dejarEnMesa(Carta c) { descarte.push(c); }
    public Carta verCima() { return descarte.peek(); }
}

```

La transición de la versión 2 a la 3 representa una evolución crítica hacia la robustez y el manejo de excepciones en la lógica del juego. La mejora principal reside en la implementación de un sistema de reabastecimiento del mazo, utilizando una pila de descarte para evitar que el juego se detenga cuando se agotan las cartas iniciales. Mientras que la versión anterior simplemente devolvía nulo, la nueva versión baraja automáticamente el descarte y lo reintegra al mazo, manteniendo la fluidez de la partida. Se optimizó la estructura de datos al usar LinkedList y Stack, lo cual es semánticamente más correcto para representar un mazo y una pila de descarte que un ArrayList genérico. Además, se aplicó un mejor encapsulamiento al marcar los atributos como privados, protegiendo la integridad de la baraja. El uso de subclasses específicas como CartaNumerica o CartaAccion en el constructor demuestra una correcta aplicación de la jerarquía de objetos definida previamente, garantizando que cada carta posea sus propiedades únicas desde su creación. En resumen, el código pasó de ser un simple generador de listas a un gestor de estado dinámico mucho más profesional y resistente a errores.

Evidencia del código 2

```
clase Jugador {
    Cadena nombre ;
    List < Carta > mano = new ArrayList <> ( ) ;

    public Jugador ( String nombre ) {
        este . nombre = nombre ;
    }

    void recibir ( Carta c ) {
        if ( c != null ) mano . add ( c ) ;
    }

    void verMano ( ) {
        Sistema . afuera . println ( " \n Mano de " + nombre + " ( " + mano . size ( ) + " cartas): " ) ;
        para ( int i = 0 ; i < mano . size ( ) ; i += 5 ) {
            int fin = Math.min ( i + 5 , mano.size ( ) ) ;
            mostrarFila ( i , fin ) ;
        }
    }

    private void mostrarFila ( int inicio , int fin ) {
        // Alineación corregida de los índices centrados sobre las cartas
        for ( int i = inicio ; i < fin ; i ++ ) {
            System.out.printf ( " [%d] " , i ) ;
        }
        Sistema.out.println ( ) ;

        for ( int linea = 0 ; linea < 7 ; linea ++ ) {
            for ( int i = inicio ; i < fin ; i ++ ) {
                String [ ] d = mano.get ( i ) . dibujar ( ) ;
                System.out.print ( d [ linea ] + " " ) ;
            }
            Sistema.out.println ( ) ;
        }
    }
}
```

Evidencia del código 3

```
// --- JUGADORES ---
abstract class Jugador {
    protected String nombre;
    protected List<Carta> mano = new ArrayList<>();
    public Jugador(String nombre) { this.nombre = nombre; }
    public void recibir(Carta c) { if (c != null) mano.add(c); }
    public String getNombre() { return nombre; }
    public void mostrarMano() {
        System.out.println("\nMano de " + nombre + " (" + mano.size() + " cartas:");
        for (int i = 0; i < mano.size(); i += 7) {
            int fin = Math.min(i + 7, mano.size());
            for (int j = i; j < fin; j++) System.out.printf(format: " [%d] ", j);
            System.out.println();
            for (int l = 0; l < 7; l++) {
                for (int j = i; j < fin; j++) System.out.print(mano.get(j).getLinea(l) + " ");
                System.out.println();
            }
        }
    }
    public abstract int elegirAccion(Scanner sc);
    public abstract int elegirCarta(Scanner sc);
    public abstract String elegirColor(Scanner sc);
}

class Humano extends Jugador {
    public Humano(String n) { super(n); }
    @Override public int elegirAccion(Scanner sc) {
        System.out.print(s: "1. Jugar | 2. Robar: ");
        try { return Integer.parseInt(sc.nextLine()); } catch (Exception e) { return 0; }
    }
    @Override public int elegirCarta(Scanner sc) {
        System.out.print(s: "Selecciona el índice: ");
        try { return Integer.parseInt(sc.nextLine()); } catch (Exception e) { return -1; }
    }
    @Override public String elegirColor(Scanner sc) {
        System.out.println(x: "Elige color: 1.Rojo 2.Verde 3.Azul 4.Amarillo");
        int op;
        try { op = Integer.parseInt(sc.nextLine()); } catch (Exception e) { op = 1; }
        String[] c = {"Rojo", "Verde", "Azul", "Amarillo"};
        return c[Math.max(a: 0, Math.min(op-1, b: 3))];
    }
}

class CPU extends Jugador {
    public CPU(String n) { super(n); }
    @Override public int elegirAccion(Scanner sc) { return 1; }
    @Override public int elegirCarta(Scanner sc) { return -1; }
    @Override public String elegirColor(Scanner sc) { return "Rojo"; }
}
```

La versión 3 introduce una mejora sustancial mediante la aplicación de clases abstractas y polimorfismo para diferenciar los tipos de participantes. Al convertir a Jugador en una clase abstracta, se define una estructura común pero se delega la toma de decisiones a subclases específicas como Humano y CPU, permitiendo que

el motor del juego trate a ambos por igual sin importar quién controle las acciones. Se mejoró la seguridad del código mediante el uso de bloques try-catch, lo que evita que el programa colapse ante entradas de usuario inválidas o no numéricas. La nueva estructura facilita la escalabilidad, permitiendo añadir diferentes niveles de dificultad o nuevos tipos de jugadores con solo extender la clase base. También se optimizó la lógica de visualización en mostrarMano y se utilizaron modificadores de acceso protected para que las subclases gestionen sus propios recursos. En esencia, el código pasó de ser un modelo rígido a un sistema flexible y tolerante a fallos que separa correctamente la interfaz de usuario de la lógica automatizada de la inteligencia artificial.

Evidencia del código 2

```
clase Juego {
    Lista < Jugador > jugadores = new ArrayList <> ( );
    Baraja mazo = nuevo Baraja ( );
    Carta mesa;
    Cadena colorActual;
    int turno = 0;
    Scanner sc = new Scanner ( System . in );

    registrador nulo ( String nombre ) {
        // Modalidad Humano vs Computadora
        jugadores . add ( nuevo Jugador ( nombre ));
        jugadores . agregar ( nuevo Jugador ( " CPU " ));
    }

    void repartir ( ) {
        para ( Jugador j : jugadores )
            para ( int i = 0 ; i < 7 ; i ++ ) j . recibir ( mazo . repartir ( ));

        boolean cartaValida = false;
        mientras ( ! cartaVálida ) {
            mesa = mazo . repartir ( );
            if ( mesa . valor . matches ( " \\ d " ) && ! mesa . color . igual ( " Negro " ) ) {
                cartaValida = verdadero;
            } demás {
                mazo . cartas . agregar ( 0 , mesa );
            }
        }
        colorActual = mesa . color;
    }

    booleano esValida ( Carta c ) {
        return c . color . equals ( colorActual ) || c . valor . equals ( mesa . valor ) || c . color . equals ( " Negro " )
    }
}
```

```

void aplicarEfecto ( Carta c ) {
    // Regla: Reversa con 2 jugadores funciona como Salto
    if ( c . valor . es igual a ( " salto " ) || c . valor . es igual a ( " rev " ) ) {
        avanzar ( ) ;
        Sistema . afuera . println ( " ¡El siguiente jugador pierde su turno! " ) ;
    } else if ( c . valor . equals ( " +2 " ) ) {
        avanzar ( ) ;
        Jugador j = jugadores . obtener ( turno ) ;
        j . recibir ( mazo . repartir ( ) ) ; j . recibir ( mazo . repartir ( ) ) ;
        Sistema . afuera . println ( j.nombre + " roba 2 cartas y pierde turno. " ) ;
    } else if ( c . valor . equals ( " +4 " ) ) {
        avanzar ( ) ;
        Jugador j = jugadores . obtener ( turno ) ;
        para ( int i = 0 ; i < 4 ; i ++ ) j . recibir ( mazo . repartir ( ) ) ;
        Sistema . afuera . println ( j.nombre + " roba 4 cartas y pierde turno. " ) ;
    }
}

void avanzar ( ) {
    turno = ( turno + 1 ) % jugadores . tamaño ( ) ;
}
}

```

Evidencia del código 3

```

// --- LOGICA DE JUEGO ---
class Juego {
    private List<Jugador> jugadores = new ArrayList<>();
    private Mesa mesa = new Mesa();
    private int turno = 0;
    private int direccion = 1;
    private String colorActual;
    private Scanner sc = new Scanner(System.in);

    public Juego(String nombreH, int cpus) {
        jugadores.add(new Humano(nombreH));
        String[] nombresCPUs = {"Pepe", "Toña", "Marix"};
        for (int i = 0; i < cpus && i < nombresCPUs.length; i++) jugadores.add(new CPU(nombresCPUs[i]));

        for (Jugador j : jugadores) for (int i = 0; i < 7; i++) j.recibir(mesa.robar());

        Carta inicio;
        do { inicio = mesa.robar(); } while (inicio.getValor().equals(anObject: "+4"));
        mesa.dejarEnMesa(inicio);
        colorActual = inicio.getColor().equals(anObject: "Negro") ? "Blanco" : inicio.getColor();

        System.out.println(x: "\n--- ¡LA PRIMERA CARTA ES! ---");
        for (int i = 0; i < 7; i++) System.out.println(inicio.getLinea(i));

        if (!inicio.getValor().matches(regex: "\\d")) {
            System.out.println(x: "¡Efecto de carta inicial activado!");
            if (inicio.getValor().equals(anObject: "Rev")) {
                direccion = -1;
                turno = (0 + direccion + jugadores.size()) % jugadores.size();
                System.out.println("¡Cambio de dirección! Inicia: " + jugadores.get(turno).getNombre());
            } else {
                inicio.aplicarEfecto(this);
            }
        } else {
            colorActual = inicio.getColor();
        }
    }
}

```

```

public void iniciar() {
    while (true) {
        Jugador actual = jugadores.get(turno);
        imprimirEstado(actual);

        if (actual instanceof Humano) turnoHumano((Humano) actual);
        else turnoCPU((CPU) actual);

        // REGLA UNO MANUAL PARA EL USUARIO
        if (actual.mano.size() == 1) {
            if (actual instanceof Humano) {
                System.out.print(s: "\n¡TE QUEDA UNA CARTA! ¡ESCRIBE 'UNO' RÁPIDO!: ");
                String grito = sc.nextLine();
                if (!grito.equalsIgnoreCase(anotherString: "UNO")) {
                    System.out.println(x: "¡No dijiste UNO correctamente! Penalización de +2 cartas.");
                    actual.recibir(mesa.robar());
                    actual.recibir(mesa.robar());
                } else {
                    System.out.println(x: "¡Perfecto! Has cantado UNO.");
                }
            } else {
                System.out.println("\n" + actual.getNombre() + " grita UNO!");
            }
        }

        if (actual.mano.isEmpty()) {
            System.out.println(x: "\n*****");
            System.out.println("¡EL GANADOR ES: " + actual.getNombre() + "!");
            System.out.println(x: "*****");
            break;
        }
        avanzar();
    }
    sc.close();
}

private void imprimirEstado(Jugador j) {
    System.out.println(x: "\n=====");
    System.out.println("TURNO ACTUAL: " + j.getNombre());
    System.out.println(x: "CARTA EN MESA:");
    for (int i = 0; i < 7; i++) System.out.println(mesa.verCima().getLinea(i));
    String valMesa = mesa.verCima().getValor().equals("Color") ? "comodin" : mesa.verCima().getValor();
    System.out.println("ESTADO -> Color: " + colorActual + " | Valor: " + valMesa);
    if (j instanceof Humano) j.mostrarMano();
}

```

NADA HUMANO ME ES AJENO

```

private void turnoHumano(Humano h) {
    int op = h.elegirAccion(sc);
    if (op == 2) {
        Carta r = mesa.robar();
        System.out.println(x: "Robaste:");
        for (int i = 0; i < 7; i++) System.out.println(r.getLinea(i));
        System.out.print(s: "¿Deseas jugar esta carta? (s/n): ");
        if (sc.nextLine().equalsIgnoreCase(anotherString: "s") && validar(h, r)) ejecutar(h, r);
        else h.recibir(r);
    } else {
        int idx = h.elegirCarta(sc);
        if (idx >= 0 && idx < h.mano.size()) {
            Carta e = h.mano.get(idx);
            if (validar(h, e)) { h.mano.remove(idx); ejecutar(h, e); }
            else { System.out.println(x: "¡Movimiento no permitido!"); turnoHumano(h); }
        } else { System.out.println(x: "Índice fuera de rango."); turnoHumano(h); }
    }
}

private void turnoCPU(CPU c) {
    Carta pJ = null;
    for (Carta ca : c.mano) if (validar(c, ca)) { pJ = ca; break; }
    if (pJ != null) {
        c.mano.remove(pJ);
        String v = pJ.getValor().equals(anObject: "Color") ? "comodin" : pJ.getValor();
        System.out.println(c.getNombre() + " jugó: " + v + " " + pJ.getColor());
        ejecutar(c, pJ);
    } else {
        System.out.println(c.getNombre() + " no tiene cartas válidas y roba.");
        c.recibir(mesa.robar());
    }
}

private boolean validar(Jugador j, Carta c) {
    if (c.getValor().equals(anObject: "+4")) {
        for (Carta m : j.mano) if (m.getColor().equals(colorActual)) return false;
    }
    return c.esCompatible(mesa.verCima(), colorActual) || c.getColor().equals(anObject: "Negro");
}

private void ejecutar(Jugador j, Carta c) {
    mesa.dejarEnMesa(c);
    if (!c.getColor().equals(anObject: "Negro")) colorActual = c.getColor();
    c.aplicarEfecto(this);
}

```

```

public void saltarTurno() { avanzar(); System.out.println(x: "¡Turno saltado!"); }

public void reversaEfecto() {
    if (jugadores.size() == 2) saltarTurno();
    else {
        direccion *= -1;
        System.out.println(x: "¡La dirección del juego ha cambiado!");
    }
}

public void hacerRobar(int n) {
    int sig = (turno + (direccion + jugadores.size())) % jugadores.size();
    for (int i = 0; i < n; i++) jugadores.get(sig).recibir(mesa.robar());
    System.out.println(jugadores.get(sig).getNombre() + " recibe " + n + " cartas y pierde turno.");
    avanzar();
}

public void cambiarColorGlobal() {
    colorActual = jugadores.get(turno).elegirColor(sc);
    System.out.println("El color ahora es: " + colorActual);
}

private void avanzar() {
    turno = (turno + (direccion + jugadores.size())) % jugadores.size();
}
}

```

La versión 3 perfecciona la lógica de control y el dinamismo del juego al implementar un sistema de dirección variable que permite que la carta "Reversa" funcione correctamente en partidas de más de dos jugadores. Se mejoró significativamente la modularización al delegar la ejecución de efectos a las propias clases de cartas, eliminando los extensos bloques condicionales que antes saturaban la clase principal. La gestión de turnos ahora es más robusta gracias a una fórmula matemática que contempla tanto el sentido horario como el antihorario, evitando errores de índice. También se integró una regla de interacción en tiempo real al obligar al jugador a "Cantar UNO", lo que añade una capa de jugabilidad que antes no existía. El uso de métodos específicos como validar y ejecutar separa las reglas del flujo de juego, haciendo que el código sea mucho más legible y fácil de mantener. En conclusión, el sistema evolucionó de una secuencia lineal rígida a un motor de juego flexible y completo que maneja estados complejos de forma eficiente.

Evidencia del código 2

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Introduce tu nombre:");
        String nombre = sc.nextLine();

        Juego juego = new Juego();
        juego.registrar(nombre);
        juego.repartir();

        while (true) {
            Jugador actual = juego.jugadores.get(juego.turno);
            System.out.println("\n--- CARTA EN MESA ---");
            for (String l : juego.mesa.dibujar()) System.out.println(l);
            System.out.println("Color actual: " + juego.colorActual);

            if (!actual.nombre.equals("CPU")) {
                actual.verMano();
                System.out.println("1. Jugar | 2. Robar");
                int op = leerOpcion(sc, 1, 2);

                if (op == 1) {
                    System.out.println("Indice de carta:");
                    int idx = leerOpcion(sc, 0, actual.mano.size() - 1);
                    Carta elegida = actual.mano.get(idx);

                    if (juego.esValida(elegida)) {
                        procesarJugada(juego, actual, elegida, sc);
                    } else {
                        System.out.println("Carta inválida. Elige otra o roba.");
                        continue;
                    }
                } else {
                    Carta robada = juego.mazo.repartir();
                    System.out.println("\nRobaste:");
                    for (String l : robada.dibujar()) System.out.println(l);
                }
            }
        }
    }
}

```

de la Ciudad de México

NADA HUMANO ME ES AJENO

```

        System.out.println("¿Quieres jugarla? (s/n)");
        if (sc.nextLine().equalsIgnoreCase("s")) {
            if (juego.esValida(robada)) {
                procesarJugada(juego, actual, robada, sc);
            } else {
                // Regla: Si roba y La carta no es válida -> pierde turno
                System.out.println("No se puede jugar. Pierdes turno.");
                actual.recibir(robada);
            }
        } else {
            System.out.println("Carta guardada. Pierdes turno.");
            actual.recibir(robada);
        }
    }

    // Regla "UNO": Si jugador tiene 1 carta debe decir "UNO"
    if (actual.mano.size() == 1) {
        System.out.print("¡Escribe UNO! ");
        if (!sc.nextLine().equalsIgnoreCase("UNO")) {
            System.out.println("Penalización +2 por no decir UNO.");
            actual.recibir(juego.mazo.repartir());
            actual.recibir(juego.mazo.repartir());
        }
    }
} else {
    Carta paraJugar = null;
    for (Carta c : actual.mano) {
        if (juego.esValida(c)) { paraJugar = c; break; }
    }

    if (paraJugar != null) {
        System.out.println("\nCPU jugó: " + paraJugar.color + " " + paraJugar.valor);
        procesarJugada(juego, actual, paraJugar, sc);
    } else {
        System.out.println("\nCPU roba y pasa.");
        actual.recibir(juego.mazo.repartir());
    }
}
}

```

Evidencia del código 3

```

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Scanner s_init = new Scanner(System.in);
        System.out.println(x: "=====");
        System.out.println(x: "      BIENVENIDO AL JUEGO UNO      ");
        System.out.println(x: "=====");
        System.out.print(s: "Ingresa tu nombre: ");
        String n = s_init.nextLine();
        System.out.print(s: "Número de oponentes (1-3): ");
        int c = 1;
        try { c = Integer.parseInt(s_init.nextLine()); } catch (Exception e) { c = 1; }

        new Juego(n, Math.min(a: 3, c)).iniciar();
    }
}

```

La versión 3 mejora drásticamente la limpieza y organización del código al aplicar el principio de responsabilidad única, trasladando toda la lógica operativa fuera del método main. Mientras que la versión anterior saturaba la clase principal con detalles técnicos como la validación de jugadas, la inteligencia artificial de la CPU y la gestión del grito de "UNO", la nueva versión delega estas tareas a métodos

especializados y a la clase Juego. El método main ahora es extremadamente minimalista, limitándose únicamente a capturar los datos iniciales y arrancar el motor del juego mediante el método iniciar. Esta encapsulación lógica facilita el mantenimiento y la lectura del flujo principal, evitando la repetición de código y permitiendo que cada componente se encargue de su propia funcionalidad. Además, se optimizó la interacción con el usuario al centralizar las peticiones de entrada, logrando una arquitectura mucho más profesional, escalable y alineada con las mejores prácticas del desarrollo orientado a objetos.

Como conclusión, el proyecto experimentó una transformación radical al pasar de un script funcional rígido a un sistema orientado a objetos profesional, robusto y escalable. La clave del éxito en esta evolución fue la implementación de la abstracción y el polimorfismo, permitiendo que el juego gestione diferentes tipos de cartas y jugadores bajo una interfaz común, lo que simplificó drásticamente la lógica del motor principal. Se logró una separación de responsabilidades impecable: las cartas conocen sus reglas, los jugadores sus estrategias y la clase Juego solo coordina el flujo, lo que facilita enormemente el mantenimiento o la adición de nuevas funciones. Además, la incorporación de un manejo de errores mediante excepciones y el uso de estructuras de datos adecuadas como Stack para el descarte aseguraron una experiencia de usuario fluida y sin interrupciones. En definitiva, el código final no solo cumple con las reglas del UNO, sino que demuestra un dominio de los pilares de la ingeniería de software, resultando en una arquitectura limpia, modular y preparada para crecer.